

romn77/TradingAgents dev 分支代码与业务评审

执行摘要

基于 dev 分支经 GitHub ¹ 连接器读取的源码，我的总体判断是：这个仓库已经从原始 TradingAgents fork 成一个“研究分析 + 选股筛选 + 资产台账 + 交易日志 + Web Workbench + 基础权限/部署”的内部研究台 MVP，方向是对的，工程骨架也已经搭起来了。它的主要优点在于顶层目录分层清楚、数据模型已经覆盖报告/筛选/资产/交易日志四类对象、并且关键路径已有回归测试；主要短板则集中在 orchestrator/service 层过胖、权限仍是粗粒度角色 + owner scope、报告读取在 optional auth 下存在匿名回退风险，以及第三方市场数据默认走明文 HTTP IP。我的结论是：它适合内部团队使用和快速迭代，但还不适合作为“多租户公开报告广场”直接上线；如果要走公开、共享或付费路径，优先级最高的不是 UI，而是权限收口、公开副本脱敏、依赖治理和错误/日志治理。 ² ³ ⁴ ⁵ ⁶

总体架构与关键判断

仓库顶层结构本身是清晰的：核心 agent/graph/dataflows 在 `tradingagents/`，筛选管线在 `tradingagents/screener/`，Web 前后端在 `web/`，部署与运维脚本在 `scripts/` 和 `compose.prod.yml`。README 还明确写出了这个 fork 相比上游新增的能力，包括数据源路由、CN/US screener、估值输入、Web Workbench、认证/管理与本地/生产部署；生产部署预案已经指向 PostgreSQL、Redis、Nginx 和 Tencent Cloud ⁷ COS 这套组合。说明作者不是只在“做模型”，而是在往“可运营的研究工作台”推进。 ² ³ ⁸ ⁹

真正的复杂度并不在 agent graph 内核，而在 Web 后端的 orchestrator/service 层：分析任务要做请求校验、额度控制、用户上下文拼装、任务排队、报告落盘、对象存储上传、元数据写库；筛选任务要做数据源决策、manifest 注入、进度事件、结果快照；资产和日志又不仅是 CRUD，还直接反向注入研究 prompt。换句话说，当前系统的“业务聪明度”更多埋在后端编排层，而不是模型层本身，这会直接影响可维护性、权限隔离和后续商业化。以下判断以静态源码审查为主，没有对真实生产环境、数据库权限、对象存储策略和第三方 API 做运行态验证。 ⁴ ⁵ ¹⁰ ¹¹ ¹² ¹³

代码整洁度

从“目录级模块化”看，这个仓库是过关的；从“函数级复杂度与职责收敛”看，问题已经开始显性化。一个明显特征是：核心问题不是命名混乱，而是**职责堆叠与跨模块粘连**。`TradingAgentsGraph`、`runner`、`analysis_tasks`、`assets`、`trades` 这些文件都表现出“把业务决定写在 orchestrator/service 里”的倾向。另一个特征是：测试并非没有，反而已经覆盖了任务队列、筛选、配置、路径安全等主干流程；但测试策略仍然是 focused checks，没有演进成 coverage gate、lint gate 或 type-check gate。 ¹⁴ ¹⁵ ¹² ¹⁰ ¹¹ ¹⁶ ¹⁷ ¹⁸

问题	位置	严重性	影响	修复建议
可变默认参数	tradingagents/graph/trading_graph.py:34-39 , __init__(selected_analysts=[...])。 14	中	这是典型 Python 可维护性隐患；今天没副作用，不代表未来不会被修改触发共享状态。	改为 <code>None</code> ，在函数体内赋默认值。
队列容量判断与 SSE 序列化重复	web/backend/routers/tasks.py:22-75 与 web/backend/routers/screeners.py:20-89 基本重复。 4 5	中	同一套规则要改两处，容易出现分析/筛选行为漂移。	抽出 <code>queue_guard.py</code> 和 <code>sse.py</code> ，统一 active/pending/user-limit 逻辑。
报告头解析重复	web/backend/services/reports.py:12-44 与 web/backend/report_metadata.py:56-84 各自实现了标题/生成时间解析。 19 6	低	报告格式一旦调整，要双改；很容易出现索引结果和实际展示不一致。	抽成共享 parser，并以单元测试锁定格式。
orchestrator/service 过胖	web/backend/runtime/analysis_tasks.py 的 <code>run_task()</code> 、 tradingagents/runner.py 的 <code>save_report_to_disk()</code> 、 web/backend/services/assets.py 的 <code>build_portfolio_context_for_owner()</code> 都同时承担多项职责。 12 15 10	高	测试难、复用差，也让权限与审计很难精准下沉。	分出 <code>report_publisher</code> 、 <code>portfolio_context_builder</code> 、 <code>analysis_task_orchestrator</code> 等更窄的应用服务。

问题	位置	严重性	影响	修复建议
状态 schema 命名漂移	tradingagents/graph/trading_graph.py 持久化日志时写入 trader_investment_decision，但其他流程普遍使用 trader_investment_plan。trade_feedback 读取快照时又依赖前者。 ¹⁴ ²⁰	中	下游消费方要记两套 key；后续做搜索、索引、公开副本时容易踩坑。	统一 schema 名称；为旧日志提供一次性迁移脚本。
依赖与配置源不够单一	pyproject.toml 大量使用 >= 浮动版本；requirements.txt 只有 .；pyproject 里 web/dev 依赖重复；.env.example 还重复了 DEEPSEEK_API_KEY。 ¹⁸ ²¹ ²²	中	可复现性不足，环境漂移和“今天能跑、明天挂掉”的概率偏高。	以 lockfile 为生产真相源；清理重复依赖与重复 env key；在 CI 中校验一致性。
测试覆盖“不均衡”而非“没有”	已抽样到的测试集中在 tests/web/test_backend_main.py 与 tests/web/test_runner.py；README 也只列出 focused checks，pyproject 未配置 coverage/lint/type-check 门禁。 ¹⁷ ² ¹⁸	中	主干流程回归能力不错，但资产、交易日志、多租户权限矩阵、公开/私有可见性切换仍缺专测。	补 permission matrix、assets/trades、public/private visibility、migration/backfill 专项测试。

结论上，这个仓库已经有“能维护”的雏形，但要跨过“能迭代”和“能规模化”的分水岭，关键不是再添模块，而是把 orchestration、schema、依赖和测试门禁收紧。¹⁸ ¹⁵ ¹²

安全性

安全基线并非空白。源码已经做了几件正确的事：ticker 先做路径安全化，报告目录和文件内容读取都做了 traversal 防护；session token 只以哈希形式落库；登录失败按邮箱/IP/pair 限流；生产 compose 默认启用

`AUTH_MODE=required` 和 `SESSION_COOKIE_SECURE=true`。这些都说明作者知道内部工具最终会走向多人使用，而不是一次性 demo。^{23 19 24 9 16}

但高优先级风险同样很明确。参考 OWASP²⁵ 对会话、错误处理和日志最小暴露的建议，当前最大问题不是密码学实现，而是配置驱动的数据暴露、供应链/第三方链路默认值和内部错误直出。OWASP 也明确指出，`SameSite` 只能提供一部分 CSRF 缓解，用户可见错误和日志不应泄露技术细节、文件路径、商业敏感信息。

²⁶

风险	定位与复现/定位路径	优先级	修复建议
可选认证下的报告匿名可读	<code>web/backend/routers/reports.py</code> 只依赖 <code>enforce_authenticated_api_access</code> ；而 <code>web/backend/services/reports.py:229-338</code> 在 <code>AUTH_MODE!=required</code> 且 <code>current_user is None</code> 时，会回退到磁盘枚举和文件读取。 <code>web/README.md</code> 还明确把 <code>AUTH_MODE=optional</code> 描述为“保留现有 workbench 可读”。复现：设置 <code>AUTH_ENABLED=true</code> ， <code>AUTH_MODE=optional</code> ，生成任意报告后，不登录直接请求 <code>/api/reports</code> 和 <code>/api/reports/{id}/content?path=complete_report.md</code> 。 ^{27 19 3}	高 P0	非本地环境直接禁止 <code>optional</code> ；把“迁移过渡可读”改为一次性后台回填任务，不要留在线匿名回退。
默认 Massive 数据源未明文 HTTP + 裸 IP	<code>.env.example:89</code> 和 <code>tradingagents/dataflows/vendors/massive/common.py:14,28-29,54-60</code> 。未配置 <code>MASSIVE_BASE_URL</code> 时会指向 <code>http://35.209.101.63/api/v1</code> 。复现：留空环境变量，触发使用 <code>massive</code> 的美股数据调用。 ^{22 28}	高 P0	删除默认值；强制显式配置 HTTPS 域名；启动时校验 scheme 与 host。
内部错误直接返回给客户端/SSE	<code>web/backend/access.py:10-20</code> 直接 <code>detail=str(exc)</code> ； <code>web/backend/services/reports.py:333-336</code> 会把底层读文件错误原样返回； <code>web/backend/runtime/analysis_tasks.py</code> 与 <code>screener_tasks.py</code> 也会把 <code>str(exc)</code> 写入任务错误和进度事件。复现：构造无效 provider、损坏 artifact、对象存储异常或第三方 API 错误。 ^{29 19 12 13}	高 P1	对外返回稳定错误码和通用消息；详细异常只写结构化日志。
依赖漏洞治理缺口	<code>pyproject.toml</code> 基本都是 <code>>=</code> ； <code>requirements.txt</code> 只有 <code>.</code> ； <code>README</code> 的安装路径是 <code>pip install .</code> 或 <code>uv sync</code> ，但源码内未体现自动审计门禁。这里我没有确认到某个直接依赖的已知 CVE，但治理上存在“未来引入漏洞/漂移版本”的现实风险。 ^{18 21 2}	中 P1	以 lockfile 为生产安装源；在 CI 加 <code>pip-audit</code> / SBOM/ Dependabot 或等效管线。
会话防护对“公开广场/分享链接/多子域”场景不够	<code>web/backend/auth.py</code> 用 cookie session， <code>web/backend/main.py:50-56</code> 开启 <code>allow_credentials=True</code> ；当前主要依赖 <code>SameSite</code> 和 origin 配置，而没有显式 anti-CSRF token / Origin-Referer 校验。对单一前端域名尚可，但一旦引入分享页、嵌入页或多子域，会放大风险。 ^{24 8 30}	中 P2	若计划做广场、分享链接或多子域，补充 CSRF token / Origin 校验，并限制 cookie Domain/ Path。

风险	定位与复现/定位路径	优先级	修复建议
密钥管理是“项目根目录 .env + 运行期注入”模型	web/backend/app_config.py 与 auth.py 在 import 时加载 .env , services/config.py 在请求阶段把 provider key 注入进 os.environ 。这对单机 MVP 简单，但对多租户或合规环境意味着 secret scope 过宽。 31 24 32	中 P2	改到进程启动时一次性装载或使用 Secret Manager；按服务、按环境最小化暴露。

就安全优先级而言，P0 是 optional-auth 报告暴露和明文 HTTP 第三方入口；P1 是错误暴露和依赖治理；P2 才是会话/secret 管理的进一步工程化。换句话说，这个项目最需要先解决“数据能不能被不该看到的人看到”，其次才是“数据传得够不够优雅”。 19 28

业务链路 with 模块边界

当前四个模块已经形成了比较完整的业务闭环，但它们并不是彼此独立的“四个小系统”，而是一个以研究为核心的联动体：分析模块会主动读取资产上下文和日志反馈，筛选模块与分析模块共享队列和数据源治理，日志模块又会反向影响分析 prompt，最终输出的报告再由报告元数据层做可见性管理。这个设计对“单团队研究台”是加分项，对“多租户外部产品”则意味着边界必须被重新画得更硬。 4 5 10 11 12 13

下面这个图概括了关键数据流与权限流。其核心结论是：分析模块不是纯研究模块，而是“研究 + 资产上下文 + 历史交易复盘”的融合模块；因此它天然比筛选模块更敏感。相关链路来自任务路由、运行时、报告元数据、资产服务和交易日志服务。 4 5 10 11 6 33 34 20

flowchart LR

```

U[用户/前端] --> R[FastAPI 路由]
R --> A[auth/access]
A --> ANA[研究-分析]
A --> SCR[研究-筛选]
A --> AST[组合-资产]
A --> JNL[组合-日志]

ANA --> Q[task_store/Redis或本地队列]
ANA --> RUN[runner + TradingAgentsGraph]
ANA --> RM[(report_runs/report_files)]
ANA --> REP[reports目录]

SCR --> SQ[共享任务队列]
SCR --> PIPE[run_screen pipeline]
SCR --> SR[(screener_runs)]
SCR --> SA[screener artifacts]

AST --> AP[(asset_accounts/positions/snapshots)]
AST --> MDC[MarketDataClient]
AST --> PC[portfolio_context 字符串]

JNL --> TE[(trade_entries)]
JNL --> TF[.trade_feedback 文件]
```

JNL --> HF[historical trade feedback]

PC --> ANA

HF --> ANA

REP --> JNL

SQ --> ANA

模块	当前职责	上下游依赖	边界评价
研究-分析	创建任务、排队、调用 <code>TradingAgentsGraph</code> 、写报告、写报告元数据	依赖资产模块生成 <code>portfolio_context</code> ；依赖日志模块回放 <code>visible_trade_ids</code> /historical feedback；依赖共享队列与数据源治理。 4 12 10 11	业务价值最高，但边界最不干净；属于“个性化研究输出”。
研究-筛选	创建筛选任务、解析 markets/manifest、运行 <code>run_screen</code> 、保存候选与运行元数据	与分析共享任务队列和 vendor routing，但本身不消费资产/日志。 5 13 35	边界比分析清晰，但与分析共享资源池，且当前没有 visibility 维度。
组合-资产	账户/仓位/估值快照管理，并把持仓摘要拼成 prompt context	DB 原生存储；调用 <code>MarketDataClient</code> ；直接服务分析模块。 36 10 34	域模型是清楚的，但暴露给分析的接口是“字符串 prompt”，不是稳定 DTO。
组合-日志	交易记录/复盘生成/反馈回放；文件与 DB 索引混合持久化	依赖报告目录中的分析快照；反向喂给分析模块。 37 11 33 20	边界最模糊：既是日志，又是 research memory，还依赖文件系统路径。

从耦合度看，系统当前有四个单点信任：`.env` secret 与 provider 凭据、共享任务队列、PostgreSQL ownership metadata、以及磁盘/对象存储中的报告与 trade artifacts。最需要警惕的不是“模块间有调用”，而是**分析模块对资产与日志的直接 prompt 级耦合**：这会让任何分析报告天然携带用户特征，进而把权限、脱敏、公开发布都变成高风险动作。 32 38 6 10 20

权限与隔离方案

当前权限模型的现实状态是：角色只有 `admin/operator/viewer` 三层；研究、筛选、资产、日志四个模块都有 weekly usage limit；实际对象访问主要靠 `owner_user_id` 做 owner scope；报告有 `private/workspace` 两级可见性，而筛选结果没有 visibility 字段。更关键的是，`viewer` 也能进入筛选模块，分析模块创建任务也没有 module-scoped capability；换言之，现状更接近“有 owner 限制的 flat RBAC”，还不是成熟的最小权限模型。根据 NIST 39 对 RBAC 层级/约束模型的定义，下一步应该从“角色存在”升级到“角色 + 模块能力 + 约束 + 租户边界”。 24 29 40 6 35 41

我的结论很明确：“研究”栏目下的两个模块都需要用户权限划分和数据隔离，其中“分析”必须比“筛选”更严格。原因是分析任务会注入 `portfolio_context` 和历史交易反馈，保存的报告也可能包含这些上下文折射；而筛选任务虽然当前更偏公共数据驱动，但一旦用户使用自定义 manifest、筛选条件或共享候选池，它同样会沉淀可交易 alpha。另一个不能忽略的细节是：`web/README.md` 的 backfill 方案会把历史报告设成

`workspace` 并指派给 bootstrap admin；这对单用户迁移可接受，但对多租户 SaaS 明显不够。 4 10 11 15 3

方案	适用场景	优点	缺点	实现复杂度	推荐优先级
最小权限 owner-only	内部小团队、先保住隐私	改动最小；与现有 <code>owner_user_id</code> 模型最贴合；分析/筛选都能快速收口	不支持团队协作与共享	低	最高
角色模型升级为 module-scoped RBAC	同组织多岗位协作	可把 <code>analysis:create/read</code> 、 <code>screener:create/read</code> 、 <code>assets:write</code> 、 <code>journal:write</code> 分开；适合 viewer/operator/admin 演进	需要路由与 service 层全面补 capability check	中	很高
增加 <code>tenant_id</code> 的租户隔离	面向外部客户或多个业务单元	真正解决跨组织数据边界；管理员不再天然全局可见	需要改表结构、查询条件、回填脚本和审计逻辑	中高	很高
审计日志 + 分享策略	公开广场、付费分享、合规追踪	适合作为共享/发布层；能追踪谁发布、谁访问、谁撤回	不能替代 owner/tenant 隔离，只是补充	中	高

落地建议是分两步：先把分析/筛选/资产/日志四个模块统一做成 **owner-only + admin override**，再往 **module-scoped RBAC + tenant_id** 演进。对“研究”栏目而言，我不建议先做复杂的角色树，而是先收紧默认值：分析结果默认私有；筛选结果默认私有，但允许将脱敏后的候选摘要升级为 **workspace/public**。 6 35

报告广场与实施路线

“报告广场”是可行的，而且不必从“公开完整报告”开始。技术上，这个仓库已经有两个很好的起点：一是 `save_report_to_disk()` 会生成 `artifacts/summary.json` 与 `artifacts/thesis.json`；二是 `report_metadata` 已经提供 report file index、ticker、generated_at 与 visibility。换言之，最快的广场 MVP 不是暴露 `complete_report.md`，而是先把**公开摘要卡片**做出来：摘要、ticker、日期、信号、作者/组织、标签、是否付费可见。需要强调的是，`trade_feedback.json` 和任何可能映射用户仓位/交易偏好的内容，不应进入公共副本。 15 19 6 20

如果要做公开模式，建议把“发布”设计成**从私有报告生成一个脱敏的公开副本**，而不是直接改原报告的 visibility。这样可以避免把文件路径、owner、portfolio context、historical trade reviews、内部错误与 vendor 细节直接带到 public surface。OWASP 的日志与错误处理建议同样适用于发布产物：公开视图中不应包含内部路径、会话值、商业敏感信息和不必要的技术细节。 19 6 42

模式	可行性	隐私/合规影响	商业价值	实现建议	推荐
匿名公开摘要	高	风险最低，只要去掉 owner、trade feedback、资产影子	最适合拉新和 SEO	仅发布 <code>summary/thesis</code> + 标签 + ticker/date	最高
署名公开报告	中	需要作者授权与组织归属	有助于建立研究品牌	新增 publisher profile、组织字段、撤稿流程	高
组织共享	高	与当前 <code>workspace</code> 最接近，但要补 tenant 边界	适合 B2B 团队协作	<code>workspace</code> 改为 tenant-scoped，不要全局 workspace	很高
付费全文/深度版	中	需审计、计费、风控、版权与撤回机制	变现空间最大	先做公开摘要 + 私有完整版，再加计费墙	中高
私链分享	高	风险可控，但要签名 URL、过期、撤回	适合销售/试用/客户沟通	<code>unlisted</code> visibility + tokenized share link	高

结合前面的代码与权限现状，我建议把实施计划分成三段：

周期	任务清单	预估工作量	主要风险
短期	强制非本地环境禁用 <code>AUTH_MODE=optional</code> ；删除 Massive 明文 HTTP 默认值；对外隐藏内部错误细节；抽取共享队列/SSE 工具；补“匿名读报告”“public/private visibility”“assets/trades 权限矩阵”回归测试	5-8 个工程日	改动虽小，但会暴露历史迁移脚本与现有前端假设
中期	引入 module-scoped permissions；为 <code>report_runs</code> 增加 <code>public/unlisted/paid</code> 可见性；给所有核心表补 <code>tenant_id</code> ；把“发布公共副本”做成独立 pipeline；给 trade journal 与 report publication 建统一审计日志	4-8 周	数据迁移、索引回填、前后端契约变化
长期	上线报告广场：摘要流、搜索、排序、标签、作者页、私链分享、付费全文；拆分 analysis 与 screener 的队列/预算/限额治理；将对象存储、审计、分享和计费真正产品化	3-6 个月	一旦对外，权限与脱敏缺陷会从“工程问题”变成“产品事故”

这条路线的关键不是一次性做“大而全”的平台，而是先把**私有研究**、**团队共享**、**公共摘要**、**付费全文**四层能力解耦：当前代码最适合先做“私有研究 + 公共摘要广场”的组合，而不是直接公开完整报告。 15 6 35

17 test_runner.py

https://github.com/romn77/TradingAgents/blob/dev/tests/web/test_runner.py

2 README.md

<https://github.com/romn77/TradingAgents/blob/dev/README.md>

3 README.md

<https://github.com/romn77/TradingAgents/blob/dev/web/README.md>

- 4 **tasks.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/routers/tasks.py>
- 5 25 **screeners.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/routers/screeners.py>
- 6 **report_metadata.py**
https://github.com/romn77/TradingAgents/blob/dev/web/backend/report_metadata.py
- 7 26 30 **https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html**
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- 8 **main.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/main.py>
- 9 **compose.prod.yml**
<https://github.com/romn77/TradingAgents/blob/dev/compose.prod.yml>
- 10 **assets.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/services/assets.py>
- 11 **trades.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/services/trades.py>
- 12 **analysis_tasks.py**
https://github.com/romn77/TradingAgents/blob/dev/web/backend/runtime/analysis_tasks.py
- 13 **screener_tasks.py**
https://github.com/romn77/TradingAgents/blob/dev/web/backend/runtime/screener_tasks.py
- 14 **trading_graph.py**
https://github.com/romn77/TradingAgents/blob/dev/tradingagents/graph/trading_graph.py
- 15 **runner.py**
<https://github.com/romn77/TradingAgents/blob/dev/tradingagents/runner.py>
- 16 **test_backend_main.py**
https://github.com/romn77/TradingAgents/blob/dev/tests/web/test_backend_main.py
- 18 **pyproject.toml**
<https://github.com/romn77/TradingAgents/blob/dev/pyproject.toml>
- 19 **reports.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/services/reports.py>
- 20 39 **trade_feedback.py**
https://github.com/romn77/TradingAgents/blob/dev/tradingagents/trade_feedback.py
- 21 **requirements.txt**
<https://github.com/romn77/TradingAgents/blob/dev/requirements.txt>
- 22 **.env.example**
<https://github.com/romn77/TradingAgents/blob/dev/.env.example>
- 23 **ticker_symbols.py**
https://github.com/romn77/TradingAgents/blob/dev/tradingagents/ticker_symbols.py
- 24 **auth.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/auth.py>

- 27 **reports.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/routers/reports.py>
- 28 **common.py**
<https://github.com/romn77/TradingAgents/blob/dev/tradingagents/dataflows/vendors/massive/common.py>
- 29 **access.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/access.py>
- 31 **app_config.py**
https://github.com/romn77/TradingAgents/blob/dev/web/backend/app_config.py
- 32 **config.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/services/config.py>
- 33 **trade_entries.py**
https://github.com/romn77/TradingAgents/blob/dev/web/backend/trade_entries.py
- 34 **asset_entries.py**
https://github.com/romn77/TradingAgents/blob/dev/web/backend/asset_entries.py
- 35 **screener_runs.py**
https://github.com/romn77/TradingAgents/blob/dev/web/backend/screener_runs.py
- 36 **assets.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/routers/assets.py>
- 37 **trades.py**
<https://github.com/romn77/TradingAgents/blob/dev/web/backend/routers/trades.py>
- 38 **task_store.py**
https://github.com/romn77/TradingAgents/blob/dev/web/backend/runtime/task_store.py
- 40 **analysis_limits.py**
https://github.com/romn77/TradingAgents/blob/dev/web/backend/analysis_limits.py
- 41 **<https://www.nist.gov/publications/nist-model-role-based-access-control-towards-unified-standard>**
<https://www.nist.gov/publications/nist-model-role-based-access-control-towards-unified-standard>
- 42 **https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html**
https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html